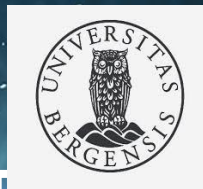


Fine-Tuning LLMs with Multi-GPU Training in an HPC System



Norwegian research infrastructure services

Hicham Agueny, PhD
Scientific Computing Group
Info. Tech. Department, UiB/NRIS



PART II

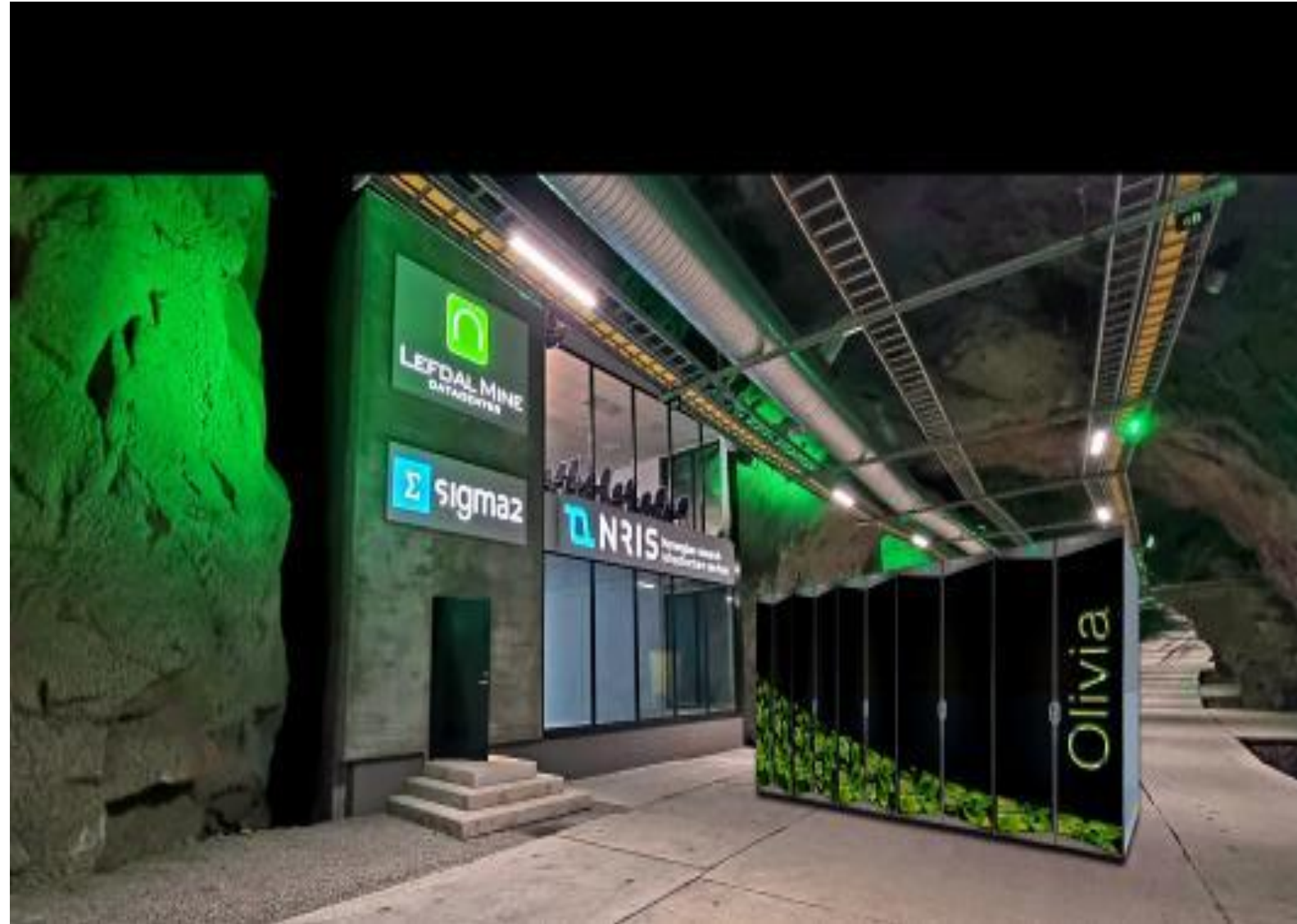
Environment Setup on Olivia Supercomputer

- **Overview of Olivia supercomputer**
- **SSH setup on Olivia**
- **Setting up PyTorch, Torchao, Torchtune & peft**
- **Verifying CUDA support** and GPU availability

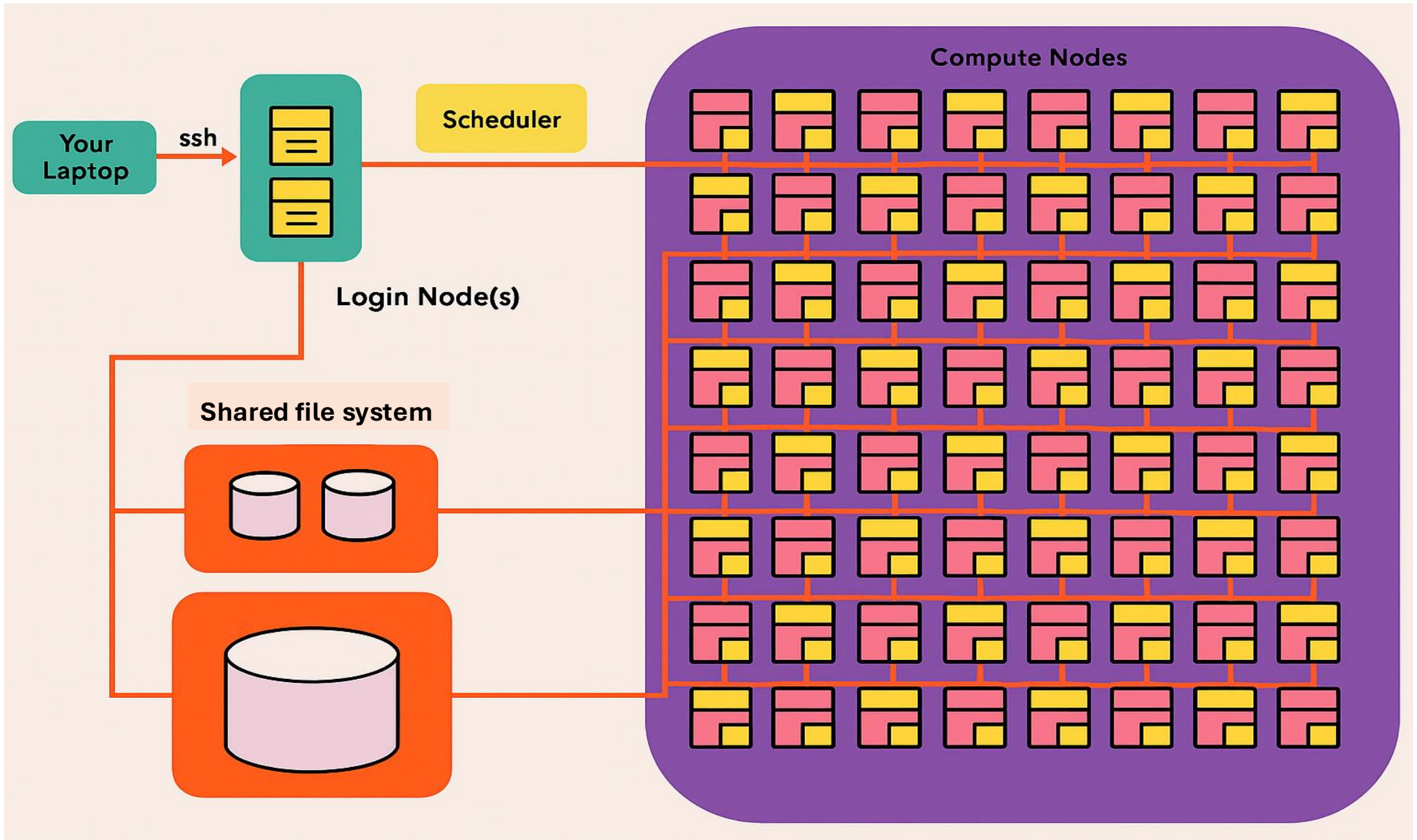


Norway's New Supercomputer - Olivia

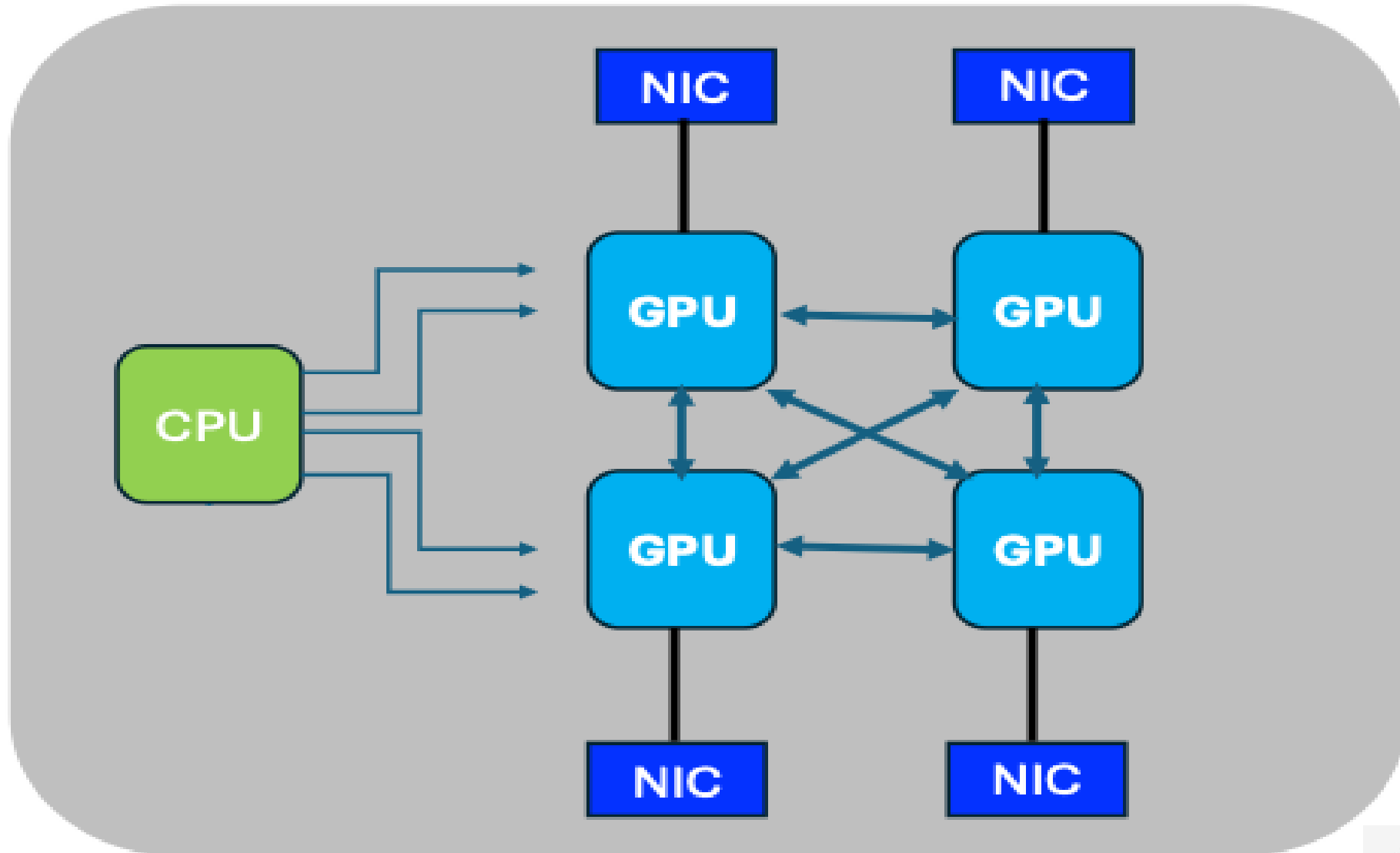
- **Olivia** is located in Lefdal Mine (Kjølsdalen)
- System: **HPE Cray Supercomputing EX**
- The GPU partition consists of:
 - 76 GPU-nodes (**304 GPUs**)
 - **NVIDIA Grace Hopper Superchip**
(**GH200 96 GB**)
 - **HPE Slingshot Interconnect**



Basic Cluster-Architecture



Compute node diagram (Simplified)



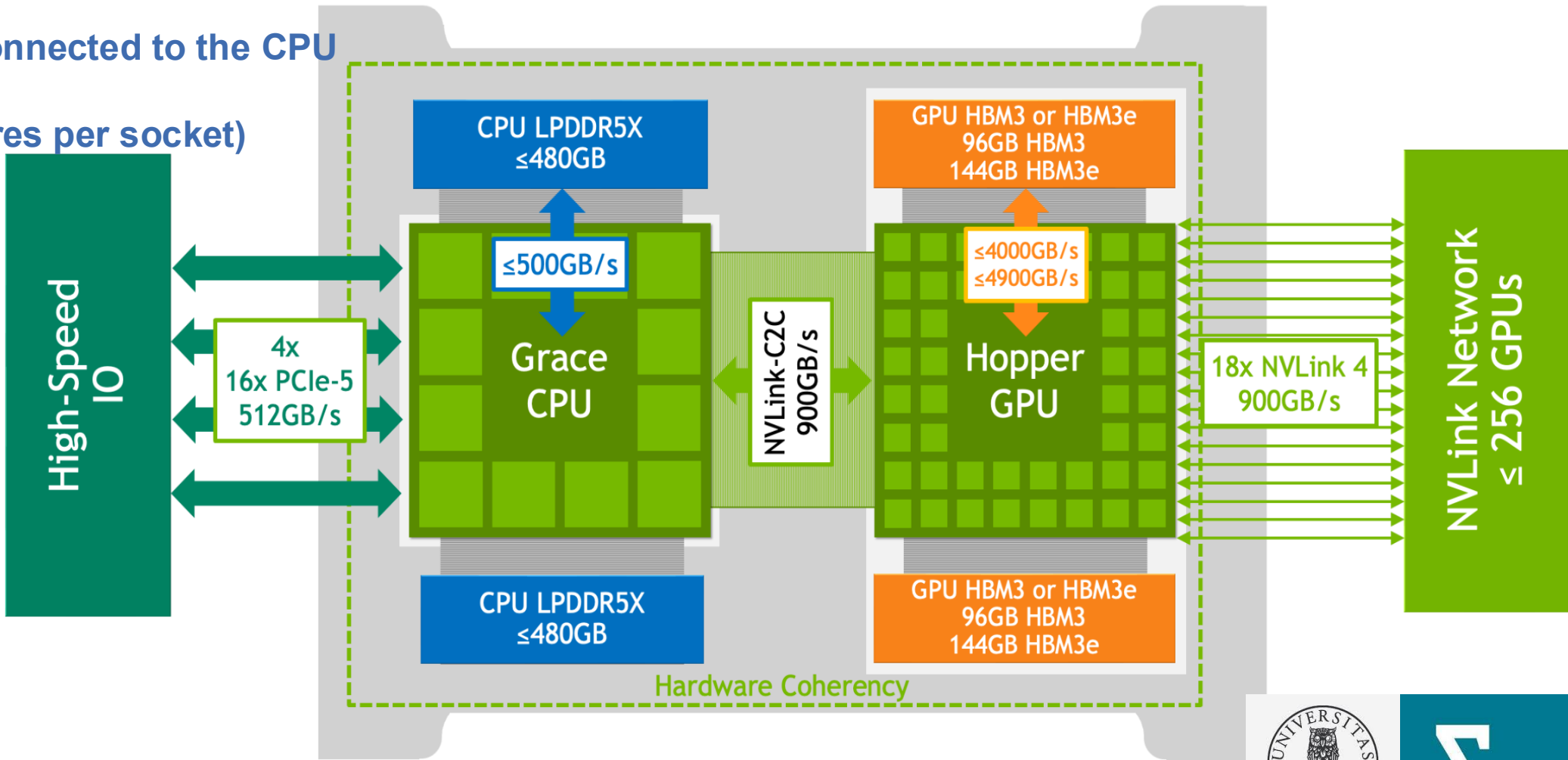
Overview of the Grace Hopper System

Olivia:
Each GH200 device has 96 GB HBM

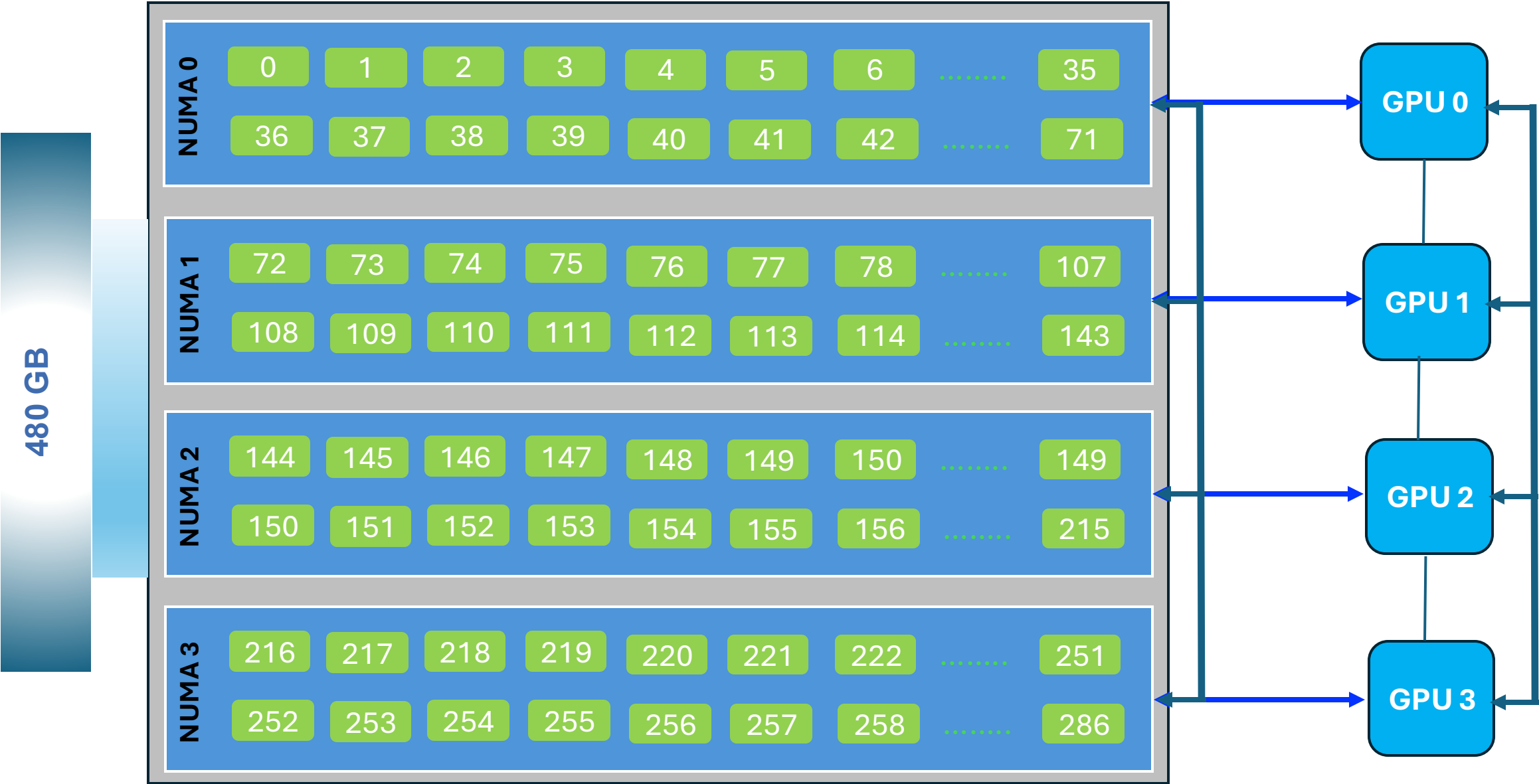
NVIDIA GH200 Grace Hopper Superchip

120 GB LPDDR5x connected to the CPU

288 cpu-core (72 cores per socket)



CPU-to-GPU affinity - (simplified diagram)



CPU-to-GPU affinity: Olivia system

- Binding a process or thread to a specific GPU.
- Workload is consistently executed on the specified GPU(s).
- Improve performance by reducing data transfer overhead.

```
hicham@x1000c0s1b1n0:~> nvidia-smi topo --matrix
GPU0      GPU1      GPU2      GPU3      CPU Affinity  NUMA Affinity  GPU NUMA ID
GPU0      X         NV6       NV6       NV6       0-71    0           4
GPU1      NV6       X         NV6       NV6       72-143   1          12
GPU2      NV6       NV6       X         NV6       144-215  2          20
GPU3      NV6       NV6       NV6       X         216-287  3          28
```

GPU	CPU Affinity	NUMA Affinity	GPU NUMA ID
GPU0	0-71	0	4
GPU1	72-143	1	12
GPU2	144-215	2	20
GPU3	216-287	3	28

`--cpu-bind=map_cpu:...` → explicitly binds each task to the CPU cores nearest its GPU.

SLURM will then match **rank 0** → **GPU0 (cores 0-71)**, **rank 1** → **GPU1 (cores 72-143)**, ... & so on.

File System Storage

Divide storage into several filesystems: **/cluster**; **/scratch**; **/flash**

Spaces can be physically or logically separated

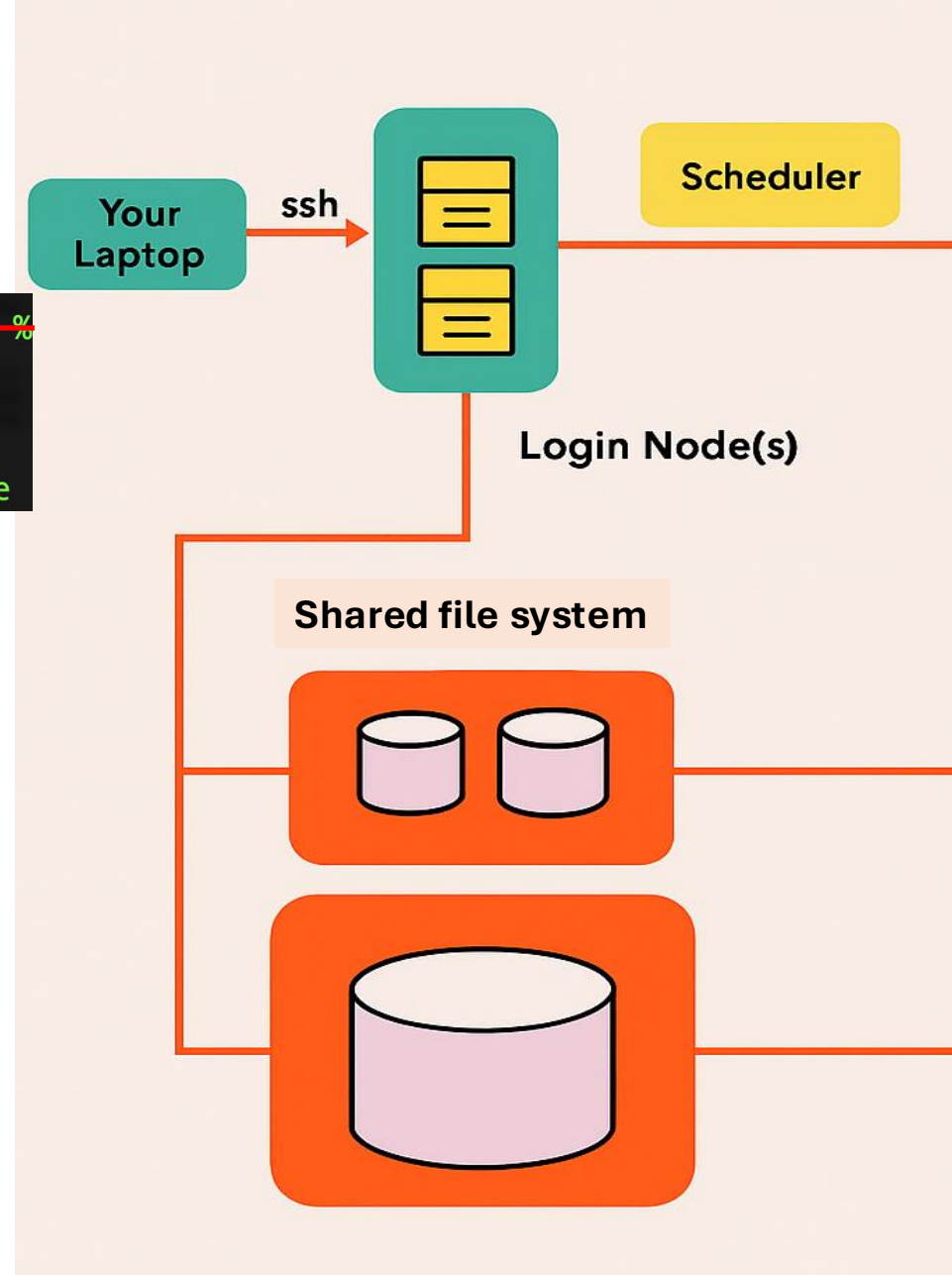
```
hicham@uan02:~> df -H -t lustre | awk '{sub(/.*:\/, "/", $1); printf "%-25s %8s %"
Filesystem                Size      Used    Avail  Use% Mounted
/cluster                   3.2P       39T     3.2P    2% /cluster
/scratch                   1.2P      128T     1.1P   12% /cluster/work
/flash                     524T      384G     519T    1% /cluster/software
```

Persistent Project Storage: /cluster

- **Purpose:** Shared, long-term files for your entire project lifecycle.
- **Policy:** Data is **meant to stay** until the project ends.

High-Speed Scratch: /scratch

- **Purpose:** **Temporary workspace** for active jobs (high-throughput I/O).
- **Policy:** Files are **not backed up** and are **automatically deleted** after a variable time (e.g., 30–90 days).
- **Warning:** Expect files to be purged sooner if the system is full.
Do not store critical files here.



Environment Setup on Olivia Supercomputer

➤ SSH setup on Olivia



SSH setup to Olivia

```
$ssh username@olivia.sigma2.no
```

```
Out:(hicham@login.olivia.sigma2.no) One-time password (OATH) for `hicham':
```

```
Out:(hicham@login.olivia.sigma2.no) Password:
```

```
$mkdir /cluster/work/projects/nn9970k/$USER
```

```
$cd /cluster/work/projects/nn9970k/$USER
```

```
$ git clone https://github.com/Hicham/llm-workshop.git
```

```
$cd llm-workshop
```

```
$./script.sh
```

Instructions

 [llm-workshop / README.md](#)

Hands-On Workflow

1. Explore SLURM job scripts.
2. Review configuration files for LoRA FT.
3. Submit jobs and monitor GPU usage.
4. Inspect logs, metrics, and visualize results.
5. Experiment with hyperparameters (e.g. LoRA rank, dropout).
6. Perform inference on QA & summarization tasks.

Detailed instructions for each step are provided in the corresponding folder README files:

- Instructions for building customised PyTorch singularity container: [container/README.md](#)
- Instructions for testing the built PyTorch container: [container/README.md](#)
- Instructions for fine-tuning setup (i.e config. files, recipes etc): [container/README.md#fine-tuning-setup](#)
- Instructions for Fine-tuning on a single GPU: [fine-tuning-singlegpu/README.md](#)
- Instructions for Fine-tuning on multiple GPUs: [fine-tuning-multigpu/README.md](#)
- Instructions for inference: [inference/README.md](#)
- Instructions for GPU monitoring and visualizing metrics: [tools/README.md](#)
- Instructions for profiling: [profiling/README.md](#)
- Guided exercises for practice are described here: [exercise/README.md](#)

Table of Contents

- [Workshop Overview](#)
- [Setup Instructions](#)
- [Folder Descriptions](#)
-  [Hands-On Workflow](#)
- [Workshop Program](#)

Environment Setup on HPC (Olivia Supercomputer)

➤ Setting up **PyTorch**, **Torchao**, **TorchTune** & **peft**

- Instructions for building customized PyTorch singularity container: [container/README.md](#)



Environment Setup on HPC - Olivia

<https://catalog.ngc.nvidia.com/orgs/nvidia/containers/pytorch/tags>

Contents of the PyTorch container

This container image contains the complete source of the version of PyTorch in `/opt/pytorch`. It is prebuilt and installed in the default Python environment (`/usr/local/lib/python3.10/dist-packages/torch`) in the container image.

The container also includes the following:

- Ubuntu 22.04 including Python 3.10
- NVIDIA CUDA 12.6.1
- NVIDIA cuBLAS 12.6.3.1
- NVIDIA cuDNN 9.4.0.58
- NVIDIA NCCL 2.22.3
- NVIDIA RAPIDS™ 24.08
- rdma-core 39.0
- NVIDIA HPC-X 2.20
- OpenMPI 4.1.7
- GDRCopy 2.3
- TensorBoard 2.16.2
- Nsight Compute 2024.3.1.2
- Nsight Systems 2024.4.2.133
- NVIDIA TensorRT™ 10.4.0.26
- Torch-TensorRT 2.5.0a0
- NVIDIA DALI® 1.41
- nvImageCodec 0.2.0.7
- MAGMA 2.6.2
- JupyterLab 4.2.5 including Jupyter-TensorBoard
- TransformerEngine 1.10
- TensorRT Model Optimizer 0.15.1

PyTorch singularity Container

Pull Singularity container

```
$ apptainer pull --arch arm64 --disable-cache  
pytorch2.5_cu2.61_py3.10.sif  
docker://nvcr.io/nvidia/pytorch:24.09-py3
```

The flag “--arch” specifies the architecture “arm64”
The flag “--disable-cache” forces a clean and fresh download.

<https://docs.nvidia.com/deeplearning/frameworks/pytorch-release-notes/rel-24-09.html>

Definition file - Olivia

Customizing the PyTorch Container

A **Singularity definition file** is a recipe for building a container.

It specifies:

- **Bootstrap source** (base image)
- **%post** section for setup commands
- Optional metadata, environment variables, and file additions

Bootstrap: localimage

Base image: Local Singularity image

From: /cluster/work/projects/nn9970k/hicham/llm-workshop/container/pytorch2.5_cu2.6.1_py3.10.sif

%post

Create /opt/python with symlinks for easy Python access.

mkdir /opt/python

ln -s /usr/bin/python3.10 /opt/python/python

Install additional Python packages inside container

python -m pip install torchtune==0.4.0

python -m pip install torchao==0.12.0

python -m pip install peft==0.17.0

```
$ apptainer build --ignore-fakeroot-command pytorch2.5_cu2.6.1_py3.10_custom.sif  
pytorch2.5_cu2.6.1_py3.10_arm.def
```

Libraries for LLM Training & Optimization

- **TorchTune:** PyTorch library for building, fine-tuning, and experimenting with LLMs. It provides well-defined **recipes** and **configurations**.
- **TorchAO:** PyTorch library for model optimizations (quantization, **memory efficiency**) to boost performance and fit larger models on hardware.
- **PEFT:** Hugging Face library for parameter-efficient fine-tuning (e.g., LoRA), reducing compute/storage costs and enabling efficient **weight merging with base models**.

Hands-on (10 min):

A customized PyTorch container is available at your work area

`/cluster/work/projects/nn9970k/$USER/llm-workshop/container`

Environment Setup on Olivia

- Instructions for testing the built PyTorch container: [container/README.md#testing-the-container](#)

Fine-Tuning Setup

- Instructions for fine-tuning setup (i.e config. files, recipes etc): [container/README.md#fine-tuning-setup](#)

N.B. You don't need to download the pretrained weights. They are already available at:

[/cluster/work/projects/nn9970k/\\$USER/llm-workshop/data/Llama-3.2-1B-Instruct](#)

Environment Setup on HPC - Olivia

Run singularity image

- Allocate an interactive session on the GPU node (**accel partition; Olivia**):

```
$salloc -A NN9970K -t 00:10:00 -p accel -N 1 --gpus 1 --mem-per-cpu 8G
```

- Launch PyTorch container

```
$apptainer shell --nv --no-home -B $PWD:$PWD pytorch2.5_cu2.6.1_py3.10_custom.sif
```

- Check CUDA support and other configs.

```
apptainer> From here run python
```

```
python -c "import torch; print(torch.cuda.is_available())"  
python -c "import torch; print(torch.__config__.show())"
```

- Check packages

```
apptainer> pip list
```

```
apptainer> tune ls
```

Fine-Tuning Setup

What is needed ? Download a Llama model, **Copy** Config & Recipe files

➤ After launching PyTorch container

Apptainer>

➤ Download **LLaMA 3.2-1B-Instruct** via e.g. Hugging Face

```
$tune download meta-llama/Llama-3.2-1B-Instruct
--output-dir /cluster/work/projects/nn9970k/$USER/llm-workshop/data/Llama-3.2-1B-Instruct
--ignore-patterns "original/consolidated*"
--hf-token your-hugging-face-token
```

➤ Copy **CONFIG** file

```
$tune cp llama3_2/1B_lora_single_device .
$tune cp llama3_2/1B_lora_multi_device .
$tune cp generation .
Change path: /tmp/Llama-3.2-1B-Instruct
To:/new-Path
sed -i "s|/tmp|/new-Path|g" 1B_lora_single_device.yaml
```

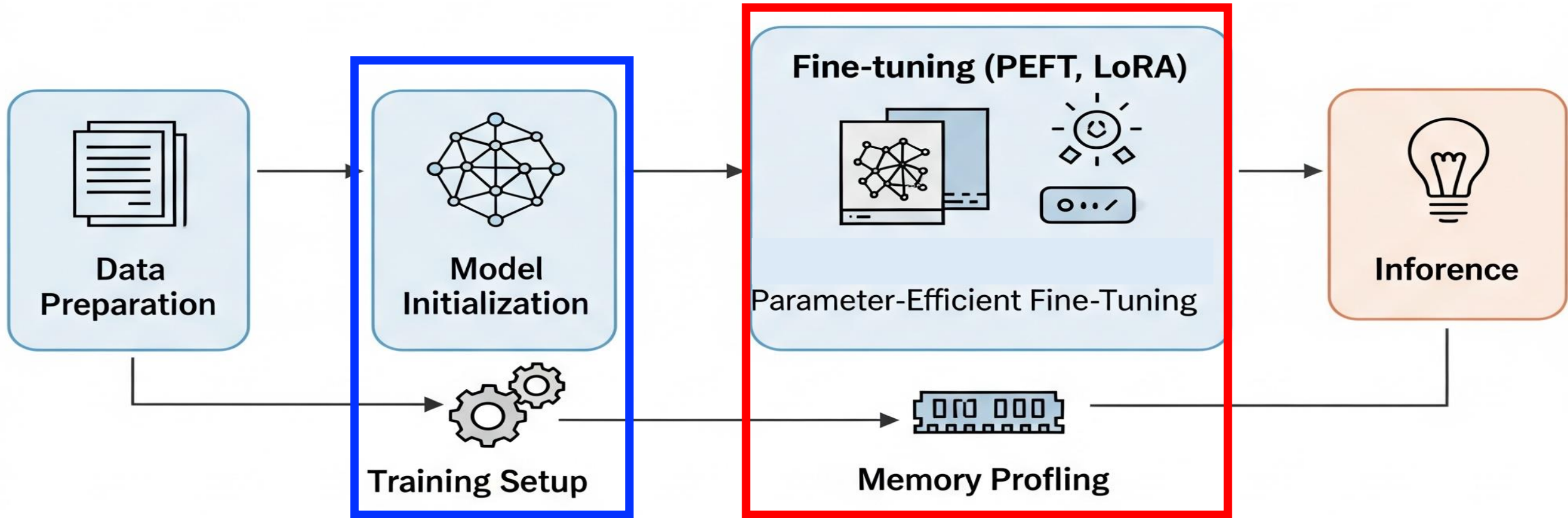
- Fine-tuning on a single GPU
- Fine-tuning on multiple GPUs
- Inference

➤ Copy built-in **RECIPE** file (python script)

```
$tune cp lora_finetune_single_device .
$tune cp lora_finetune_distributed .
$tune cp generate .
```

- Fine-tuning on a single GPU
- Fine-tuning on multiple GPUs
- Inference

Fine-tuning pipeline for LLM

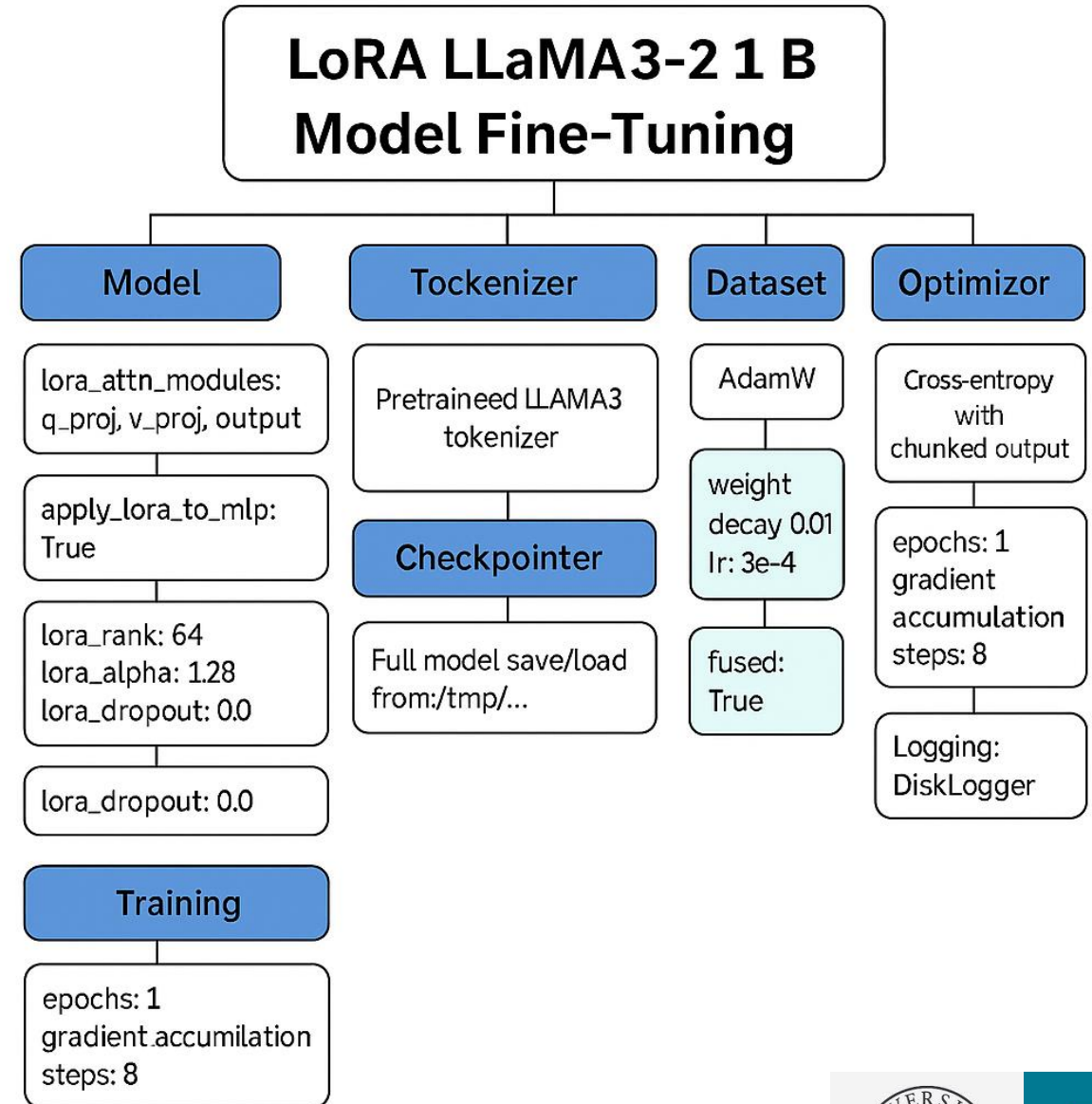


LLaMA3-2 1B Fine-Tuning Config Overview

Single GPU

Due to limited computational resources:

- Light model: 1B parameter of LLaMA3-2
- LoRA techniques for fine-tuning

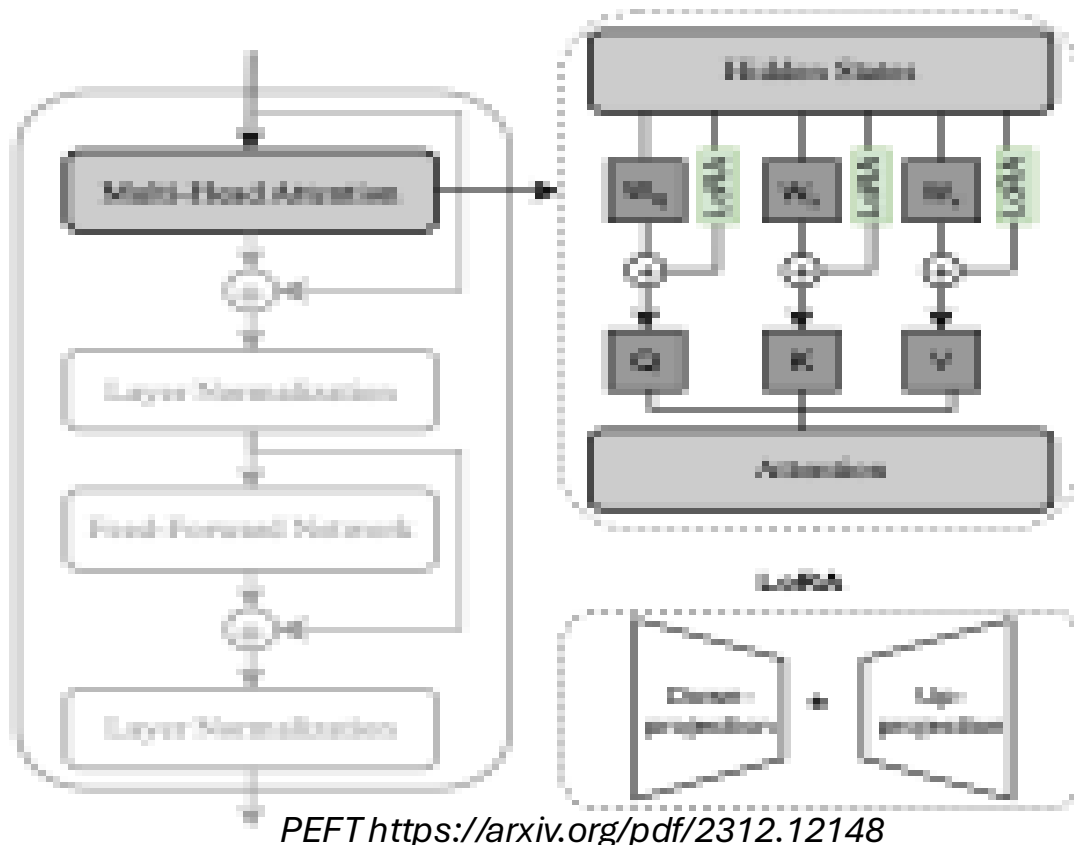


Model Arguments

```
model:
  _component_: torchtune.models.llama3_2.lora_llama3_2_1b
```

```
lora_attn_modules: ['q_proj', 'v_proj', 'output_proj']
```

- Loads the LLaMA3-2 1B model with **LoRA** integration.
It's a **1B parameter variant of LLaMA3.2** (Meta's LLaMA3 architecture)
- LoRA is applied to specific transformer attention components: **query, value, and output projections**.



PEFT <https://arxiv.org/pdf/2312.12148>

- The **Query vector** represents what information a token is trying to find.
- The **Key vector** represents what information the token contains.
- The **Value vector** carries the actual content that will be aggregated during attention.

Model Arguments

```
model:  
  _component_: torchtune.models.llama3_2.lora_llama3_2_1b
```

```
lora_attn_modules: ['q_proj', 'v_proj', 'output_proj']
```

```
apply_lora_to_mlp: True
```

```
lora_rank: 64  
lora_alpha: 128  
lora_dropout: 0.0
```

- Loads the LLaMA3-2 1B model with **LoRA** integration. It's a **1B parameter variant of LLaMA3.2** (Meta's LLaMA3 architecture)
- LoRA is applied to specific transformer attention components: **query, value, and output projections**.
- Enables **LoRA** for the model's MLP (feedforward) layers too, not just attention. (`apply_lora_to_mlp: False`: only to attention layers)
- **lora_rank** determines low-rank dimensionality of matrices **A** and **B** that are added to the pre-trained model.
- **lora_alpha** scales LoRA updates (commonly $2 \times \text{rank}$).
- **lora_dropout** is optional and is turned off (can be tuned if overfitting is noticed).



Model Arguments

```
apply_lora_to_mlp: True
```

```
lora_rank: 64  
lora_alpha: 128  
lora_dropout: 0.0
```

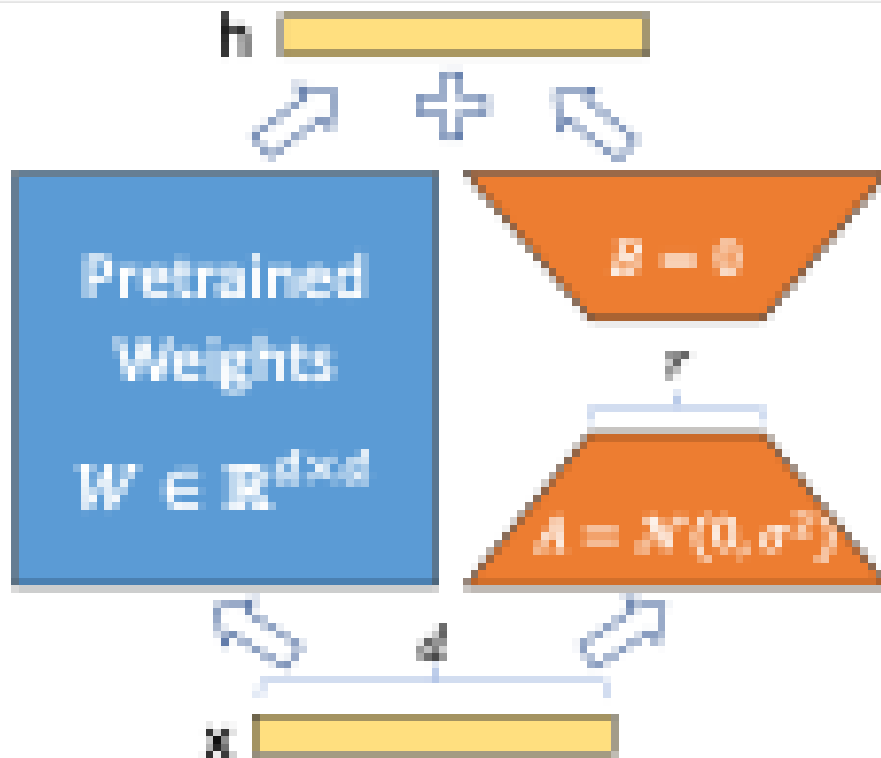
- Enables **LoRA** for the model's MLP (feedforward) layers too, not just attention. (`apply_lora_to_mlp: False`: only to attention layers)
- **lora_rank** determines low-rank dimensionality of matrices **A** and **B** that are added to the pre-trained model.
- **lora_alpha** scales LoRA updates (commonly $2 \times \text{rank}$).
- **lora_dropout** is optional and is turned off (can be tuned if overfitting is noticed).



Model Arguments

```
apply_lora_to_mlp: True
```

```
lora_rank: 64  
lora_alpha: 128  
lora_dropout: 0.0
```



LoRA <https://arxiv.org/pdf/2106.09685>

- Enables **LoRA** for the model's MLP (feedforward) layers too, not just attention. (`apply_lora_to_mlp: False`: only to attention layers)
- **lora_rank** determines low-rank dimensionality of matrices **A** and **B** that are added to the pre-trained model.
- **lora_alpha** scales LoRA updates (commonly $2 \times \text{rank}$).
- **lora_dropout** is optional and is turned off (can be tuned if overfitting is noticed).

Modified forward pass

$$h = W_0 x + \Delta W x = W_0 x + \underbrace{B A x}_{\text{LoRA update}}$$

Random Gaussian initialization for A and zero for B

$\Delta W = BA$ is zero at the beginning of training



Tokenizer

```
tokenizer:  
  _component_: torchtune.models.llama3.llama3_tokenizer  
  path: /tmp/Llama-3.2-1B-Instruct/original/tokenizer.model  
  max_seq_len: null
```

- Specifies a tokenizer built for LLaMA 3 models.
- Defines the path to the tokenizer model file.
- Specifies the maximum number of tokens the tokenizer should return
- **null** means no limit is enforced here.

Tokenizer

```
tokenizer:
  _component_: torchtune.models.llama3.llama3_tokenizer
  path: /tmp/Llama-3.2-1B-Instruct/original/tokenizer.model
  max_seq_len: null
```

- Specifies a tokenizer built for LLaMA 3 models.
- Defines the path to the tokenizer model file.
- Specifies the maximum number of tokens the tokenizer should return
- **null** means no limit is enforced here.

Checkpoint

```
checkpointer:
  _component_: torchtune.training.FullModelHFCheckpointer

  checkpoint_dir: /tmp/Llama-3.2-1B-Instruct/
  checkpoint_files: [model.safetensors]
  recipe_checkpoint: null
  output_dir: /tmp/Llama-3.2-1B-Instruct/
  model_type: LLAMA3_2
  resume_from_checkpoint: False
  save_adapter_weights_only: False
```

- Class used for checkpointing.
- Saving and restoring the full Hugging Face model.
- Directory **where checkpoints are read from**.
- Tells the checkpointer **which specific files to load**.
- Loading a training recipe or configuration from a checkpoint.
- Directory to **save** the checkpoints.
- Specifies the model type Llama-3.2-1B-Instruct.
- **False**: Starts a new training from pre-trained base otherwise it continues from where it previously stopped.
- **False**: Entire model will be saved including LoRA weights.
- **True**: only **LoRA/adaptor weights** are saved.
(It will require a separate step to merge them later with the base model at inference).

Key	Meaning
checkpoint_dir	Where checkpoints live (read/write)
checkpoint_files	File(s) to load (here: model.safetensors)
recipe_checkpoint	Optional training state checkpoint (not used here)
output_dir	Where to save new checkpoints
model_type	Tells the system how to parse model weights
resume_from_checkpoint	Whether to continue from a saved state
save_adapter_weights_only	Whether to save only LoRA weights or the full model

Dataset & Sampler

```
dataset:
  _component_: torchtune.datasets.alpaca_cleaned_dataset
  packed: False
seed: null
shuffle: True
batch_size: 4
```

- Dataset of instruction-following demonstrations based on OpenAI's text-davinci-003
- Indicates whether sequences are *packed* for training.
- **False:** Padding is used to ensure consistent sequence lengths within a batch.
- **True:** Input examples are packed together into a continuous stream, which can increase training speed by reducing padding overhead.
- No specific random seed is set.
- Dataset is shuffled to prevent the model from overfitting.
- Number of samples to process in **one step**.



Optimizer & Scheduler

optimizer:

```
_component_: torch.optim.AdamW  
fused: True  
weight_decay: 0.01  
lr: 3e-4
```

- Specifies the optimizer class: AdamW is widely used in Transformer training.
- Enables a **fused backend implementation of the optimizer** (speeds up training).
- Helps prevent overfitting.
- This is the initial learning rate.

lr_scheduler:

```
_component_: torchtune.training.lr_schedulers.get_cosine_schedule_with_warmup  
num_warmup_steps: 100
```

- Defines how the learning rate changes over time.
- Uses a built-in TorchTune utility to get a **cosine decay** schedule with warmup.
- During warmup, the LR gradually increases from zero to the target LR.
- After warmup, the LR follows a **cosine decay**, slowly decreasing over time (effective for transformer models)
- Number of steps to gradually increase the learning rate before cosine decay begins.

Loss Function

loss:

```
_component_: torchtune.modules.loss.CEWithChunkedOutputLoss
```

- Specifies the loss function used during training.
- **Cross-Entropy loss** designed to work with chunked model outputs.



Training Config

```
epochs: 1
max_steps_per_epoch: null
gradient_accumulation_steps: 8
compile: False
```

- Sets the total number of full passes over the training dataset.
- Sets the optional limit on how many steps to run **per epoch**.
- **Null:** means it will run over the **entire dataset** for each epoch.
- Sets how many steps before applying an optimizer update.
- Useful for a limited GPU memory. Number of forward + backward passes to accumulate gradients **before updating weights**.
- Allows simulating a **larger batch size** without increasing GPU memory usage.
- **False:** Disables torch.compile()
- **True:** Enables torch.compile(), which can speed up and optimize model execution.



Logging

```
output_dir: /tmp/lora_finetune_output
metric_logger:
  _component_: torchtune.training.metric_logging.DiskLogger
  log_dir: ${output_dir}
log_every_n_steps: 1
log_peak_memory_stats: True
```

- Specifies the directory where outputs (logs, metrics, etc.) will be saved
- **DiskLogger** writes logs to disk (Useful for later analysis or visualization).
- Sets the log directory to be the same as **output_dir**: /tmp/lora_finetune_output
- Logs metrics **after every single training step**.
- **True**: Enables logging of **peak GPU memory usage**.
- (useful for debugging memory issues or profiling training efficiency)

Environment

```
device: cuda
dtype: bfloat16
```

- Specifies that training will run on a **CUDA-compatible GPU**.
- Sets the computation **data type** to **bfloat16**, a lower-precision format.
- (16-bit floating-point format).
- Less precise than fp32 (32 bit) but more than fp16 (16 bit).



Activation Memory Control

```
enable_activation_checkpointing: False
enable_activation_offloading: False
```

Memory optimization Flags:

- **False:** Default. All activations checkpointings are stored (faster training, higher memory usage).
- Moves activations from **GPU to CPU** memory during training.
- **True:** Reduces GPU memory usage, but slows training. Useful for **very large models** that don't fit entirely on GPU.

Profiler Configuration

```
profiler:
  _component_: torchtune.training.setup_torch_profiler
  enabled: False
```

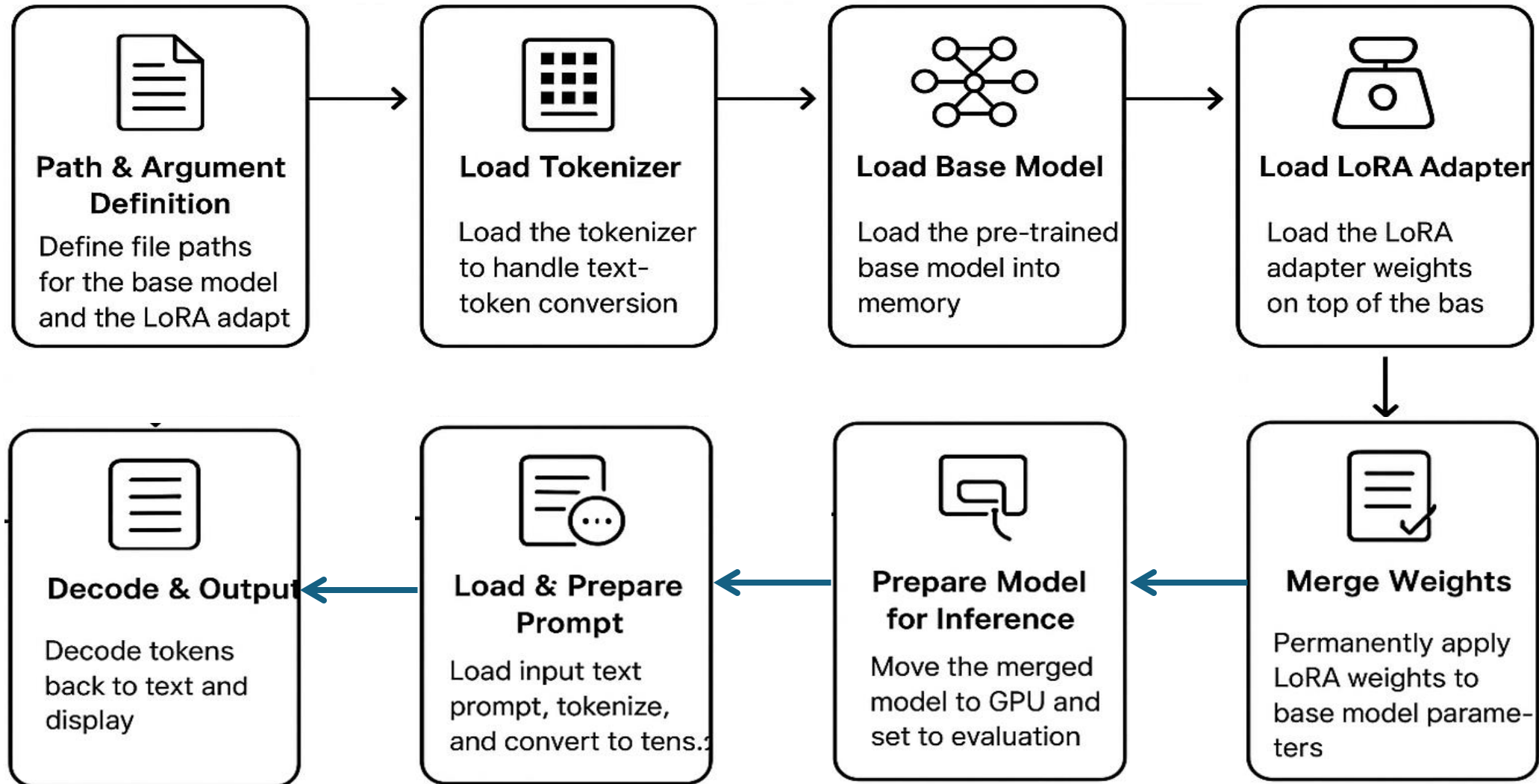
- Profile training performance (CPU/GPU utilization, memory, bottlenecks).
- False: means no profiling occurs.

```
output_dir: ${output_dir}/profiling_outputs # Saves traces (e
```

```
cpu: True # Profile CPU operations
cuda: True # Profile GPU (CUDA) operations
```

- Specifies the directory where profiling outputs are saved.
- Enables tracking of operations on CPU & GPU devices.

Structure of the Inference process



END PART II

Hands-on:

Fine-Tuning on a single GPU + Inference

Dataset: Alpaca dataset

Question Answering tasks